

Das Problem: ein Skalar verdeckt alles

Aggregierte Fitness mittelt Teilleistungen zu einer Zahl — Spezialisten verlieren gegen den soliden Durchschnitt

	Balance	Sinkflug	x-Præz.	Treibstoff	Reward	
Individuum S (Spezialist)	10	9	9	2	3	Mittel 6.6
Individuum D (Durchschnitt)	6	6	6	6	6	Mittel 6.0

S beherrscht Balance und Präzision nahezu perfekt — genau die Nische, aus der eine gute Lösung wachsen konnte. Bei aggregierter Fitness (Turnier) entscheidet aber der Mittelwert: 6.6 gegen 6.0 ist ein Wimpernschlag, und schon eine kleine Skalen-Änderung bei "Treibstoff" dreht das Ergebnis.

Folge: die Population konvergiert auf Generalisten, seltene Teilkompetenzen sterben aus, Diversität geht verloren.

Die Idee: Lexicase Selection

Spector 2012 — ein Elternteil pro Selektionsereignis, gefiltert Testfall für Testfall in zufälliger Reihenfolge

Ein Selektionsereignis (= ein Elternteil ziehen)



Warum "lexicase"?

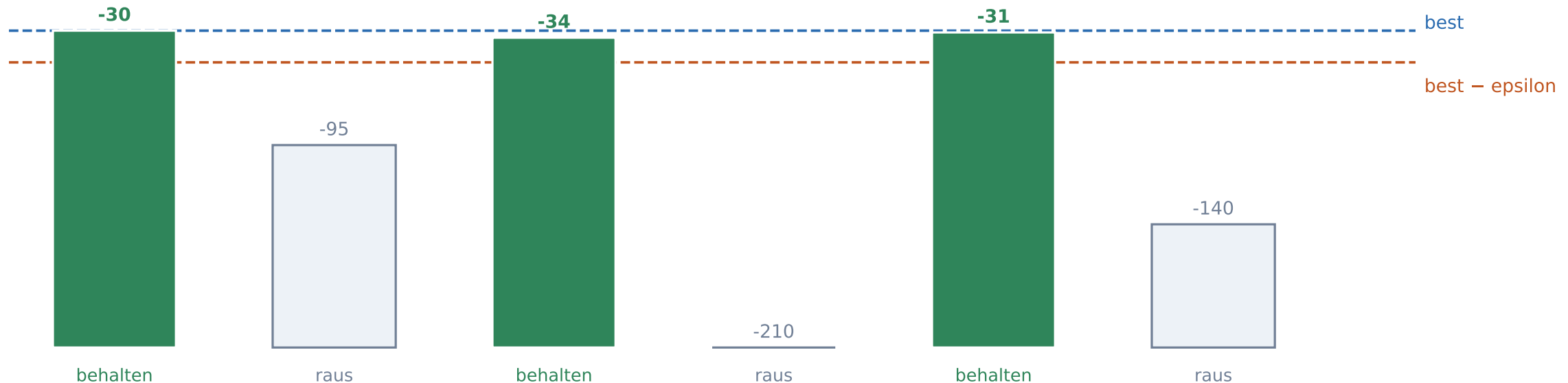
Wie bei der alphabetischen (lexikografischen) Ordnung: der erste Testfall entscheidet zuerst, der zweite nur noch die Gleichstände des ersten, usw. Weil die Reihenfolge jedes Mal neu gewürfelt wird, bekommt über viele Ziehungen jeder Testfall die Chance, "zuerst" zu sein.

Kein Mittelwert, keine Gewichte, keine Skalierung der Ziele. Wer irgendeinen Testfall am besten löst, wird mit einiger Wahrscheinlichkeit als Elternteil gewählt — auch als reiner Spezialist.

Epsilon-Lexicase für kontinuierliche Fitness

La Cava et al. 2016 — bei reellwertigem Reward ist "exakt gleich gut" nie erfüllt; eine Toleranz macht das Filtern sinnvoll

Ein Testfall, sechs Kandidaten (hoher = besser)



epsilon automatisch pro Testfall und Runde (Default, $\epsilon < 0$):

epsilon = Median der absoluten Abweichungen vom Median (MAD) über die aktiven Kandidaten — robust gegen Ausreisser, skaleninvariant, kein Handtuning. Alternativ fester Wert (epsilon ≥ 0).
Filterregel: behalte $c \iff \text{wert}(c) \geq \text{best} - \epsilon$. Pool schrumpft monoton bis 1 übrig ist.

Umsetzung im Projekt

`Population::lexicaseSelection(int E, float epsilon = -1.0f, std::string type = "objectives")`

type = "objectives" (Default)

5 isolierte Ziele aus `network.lexicaseObjectives`
(über alle Seeds gemittelt, höher = besser):

- 1 Balance `sum(-(|angle|+|angVel|))`
- 2 Sinkflug-Sicherheit `-(|vx|+|vy|)_end`
- 3 Horizontale Präzision `-|x|_end`
- 4 Treibstoff-Effizienz `sum(fuel penalty)`
- 5 Gym-Standard-Reward (holistisch)

type = "seeds"

Testfall = ein Seed aus `gymnasiumMultiSeed()`.

Pro Seed ein einziges Kriterium:

`fitnessValues[seedIdx]` (Rohreward)
maximieren, MAD- oder fester epsilon.

So im Beispiel `lunarlander.py`:

`pop.lexicaseSelection(E=1, type="seeds")`

Gemeinsame Mechanik

- Frisch geworfene Testfall-Reihenfolge pro Elternteil (`std::shuffle` mit `*generator`).
- Filtern bis `Pool <= 1`; Rest per uniform random tie-break.
- `ni` - `E` Kinder werden so gezogen; die `E` Elite-Individuen bleiben unverändert (`setElite()` nach aggregierter fitness, wie in `tournamentSelection()`).
- `bestFit` / `meanFitness` / `minFitness` werden identisch zur Turnier-Selektion geführt.

Vorbedingung: `lexicaseObjectives` (5-D) bzw. `fitnessValues` gleich gross und nicht leer —
befüllt von `gymnasiumMultiSeed()` / `gymnasium()` via `Network::fitGymnasium()`.

Eigenschaften, Kosten, Einordnung

Wann Lexicase hilft — und was es kostet

Starken

- + Erhalt Spezialisten und damit Populations-Diversität (Druck auf "corner cases").
- + Implizit mehrzielig: keine Gewichte, keine Normierung, keine Pareto-Front noting.
- + Gut bei modularen / multimodalen Problemen (Spector: "problem modality").
- + epsilon-Variante macht es auf reellwertige Rewards (Regression, RL-Return) anwendbar.

Kosten & Grenzen

- Aufwand $\sim O(\#Eltern \times \#Testfalle \times Poolgrosse)$ pro Generation.
- Braucht aussagekräftige, möglichst unkorrelierte Testfälle — hier nur 5 Ziele bzw. #Seeds.
- Bei sehr vielen quasi-identischen Testfällen degeneriert es Richtung Zufallswahl.

Einordnung im Projekt

tournamentSelection() = ein Skalar, paretoTournamentSelection() = Dominanz auf Ziel-Vektor, lexicaseSelection() = Filtern Testfall für Testfall. Selbe Schnittstelle (Elite E, Bookkeeping), untereinander austauschbar in der Evolutionsschleife.